Høgskulen på Vestlandet

Prosjektrapport i DAT255 – Deep Learning Engineering

Real or Rendered?

30.04.2026

Tobias Hanssen (kand.nr. 470)

Thomas Årland (kand.nr. 389)

Jørgen Troland (kand.nr. 487)

Embrik Børresen (kand.nr. 356)

Jeg bekrefter at arbeidet er selvstendig utarbeidet, og at referanser/kildehenvisninger til alle kilder som er brukt i arbeidet er oppgitt, jf. Forskrift om studium og eksamen ved Høgskulen på Vestlandet, § 10.

Innholdsfortegnelse

Prosjektrapport i DAT255 – Deep Learning Engineering 1
Innholdsfortegnelse 2
Problembeskrivelse 3
Data 5
Implementering av modell 6
Arkitektur 6
Klassifiseringshode 7
Treningsstrategi 8
Hyperparameter og iterasjon 9
Evaluering 10
Deployment 14
Konklusjon 16
Kilder 18
Tabbeliste: 20
Vedlegg 20

Problembeskrivelse

De siste årene har AI-modeller som Stable Diffusion, Midjourney og DALL·E gjort det enkelt å produsere fotorealistiske bilder fra tekstlige beskrivelser (OpenAI, 2021). Kvaliteten har økt så raskt at det for et utrent øye ofte er umulig å avgjøre om et gitt bilde er tatt med et kamera eller generert av en AI-modell (Storehaug, J., u.å). Dette fører til flere samfunnsmessige problemer; spredning av desinformasjon og deepfake, svekket tillit til visuell dokumentasjon i nyhetsbildet, og nye former for svindel og identitetstyveri (Norum Bugge, T., 2025). Samtidig øker etterspørselen etter automatiserte deteksjonsverktøy fra plattformer som skal moderere innhold, fra journalister som verifiserer kilder, og fra forskere som studerer epistemiske effekter av syntetisk medieinnhold.

[Bilde 1] Eksempel av et AI-generert bilde generert av Midjourney.

I dette prosjektet bygger vi et verktøy som ser på et bilde og forteller om det er ekte fotografi eller AI-generert. Løsningen er pakket som en nettapplikasjon bygget med Streamlit og deployet på Hugging Face Spaces, slik at en bruker kan laste opp et vilkårlig bilde (JPG, PNG eller WebP) og motta både en merkelapp ("Real" eller "AI-generated") og en sannsynlighetsscore mellom 0 og 1. Fire ulike modeller ligger bak applikasjonen: tre backbones finjustert fra ImageNet-forhåndstrening (ViT-B/16, DenseNet-121, ResNet-50), samt en ResNet-50 vi har trent fra bunnen av med tilfeldig initialiserte vekter og GELU-aktiveringsfunksjoner. Brukeren kan selv velge modell fra en nedtrekksliste og sammenligne prediksjoner på tvers av arkitekturer.

Tradisjonelle metoder innen bildegjenkjenning, slik som håndlagde filtre, JPEG-frekvensanalyse eller PRNU-støysignaturer, har tidligere blitt brukt for validering, men fungerer dårlig mot moderne diffusjonsmodeller. Grunnen er at artefaktene i AI-genererte bilder ikke lenger er lokale, slik som karakteristisk blokkstøy i tidligere GAN-bilder, men heller består av subtile globale inkonsistenser: anatomiske feil, umulig fysikk i refleksjoner og skygger, og finkornet teksturfordeling som avviker fra naturlige bildestatistikker (Lilleslåtten, M., 2023).

Det finnes allerede eksisterende kommersielle og forskningsbaserte løsninger, slik som Hive Moderation (Hive AI, u.å), Optic "AI or Not" (Optic, u.å) og Sensity AI (Sensity AI, u.å), samt åpne verktøy fra akademiske miljøer (for eksempel Wang m.fl., 2019; Corvi m.fl., 2023). Vårt bidrag er ikke primært å slå de beste publiserte modellene i feltet, men å dokumentere en komplett end-to-end pipeline fra datainnsamling og augmentering, via sammenligning av ulike arkitekturstrategier, til deployment i en webapp, på et nivå som samsvarer med målsetningen i DAT255 om å beherske ingeniøraspektene ved dyp læring.

Som suksesskriterier bruker vi standard binære klassifikasjonsmetrikker: AUC (ROC) som overordnet rangeringsmål, accuracy som grovt nøyaktighetsmål, samt precision, recall og F1-score for å sikre at modellen ikke bare er skjevt god på én av klassene. En målsetning definert før trening var å oppnå AUC ≥ 0.90 for vår egentrente modell og AUC ≥ 0.95 for minst én finjustert modell. I tillegg rapporterer vi treningstid, modellstørrelse og inferenstid som praktiske metrikker knyttet til deployment.

Accuracy	Måler andelen riktig predikasjoner totalt
Precision	Hvor stor andel av bilder merket AI som faktisk er det.
Recall	Hvor stor andel av de faktisk AI-bildene modellen fanger opp
F1	Det harmoniske gjennomsnittet mellom Recall og Precision. Nyttig når en vil balansere begge feiltypene
AUC	Måler modellens evne til å rangere AI-bilder over ekte bilder på tvers av alle mulige terskler

Tabell 1: Forklaring av måleparameter.

Data

Datasettet leveres ferdig delt fra Kaggle i 48 000 treningsbilder og 12 000 testbilder (Zhang, 2024). De 48 000 treningsbildene splittet vi videre med en stratifisert 80/10/10-fordeling (seed=42), slik at 38 400 bilder ble brukt til trening, 4 800 til validering, og 4 800 til et internt testsett som vi brukte under utvikling og modellseleksjon. Den endelige evalueringen i Del 4 er utført på de 12 000 originale Kaggle-testbildene, som modellene aldri har sett under verken trening, validering eller modellseleksjon, og som derfor gir et strengt og upartisk estimat på reell ytelse.
En viktig metodisk beslutning er at alle fire modellene (ResNet-50 trent fra bunnen av, samt ResNet-50, DenseNet-121 og ViT-B/16 finjustert fra ImageNet) bruker nøyaktig samme datasplitt og samme preprosesseringspipeline. Det gir en kontrollert sammenligning der den eneste forskjellen mellom eksperimentene er arkitektur og initialisering, noe som gjør det mulig å tilskrive forskjeller i ytelse til modellvalget og ikke til tilfeldige variasjoner i dataoppdelingen.

Det er flere fordeler ved datasettet som gjør det egnet for dyp læring. En fordel er størrelsen. Den er tilstrekkelig stor til at modeller med mange millioner parametere kan trenes uten at de må overtilpasses, dersom regulariseringen settes fornuftig. Klassebalansen er 1 til 1, noe som forenkler metrikvalg siden accuracy blir meningsfull og det er ikke behov for å vekte klassene under trening. I datasettet dekker bildene et bredt spekter av motiver, alt fra portretter og landskap, til gjenstander. Dette gjør at modellene må lære generelle kjennetegn på AI-generering og ikke motivspesifikke korrelasjoner.

Datasettet har også begrensninger. Bildene er skalert til 224x224 piksler før modellene ser dem. Dette gjør pipelinen kompatibel med standard ImageNet-basert arkitektur. Det betyr også at eventuelle høyoppløste artefakter som bare er synlige i høyere oppløsning kan forsvinne.
Det finnes alternative datasett. For eksempel FaceForensics++(spesialisert på deepfake-ansikter) og CIFAKE (lavoppløste 32x32 bilder)(Bird & Lofti, 2023). Datasettet som ble brukt er valgt fordi det har en god balanse mellom størrelse, oppløsning og motiv.
Vi bruker både forhåndstrente modeller og en modell som vi laget selv. De forhåndstrente modellene er alle trent på ImageNet-1k, som er et standard benchmark med ca 1.28 millioner bilder, fordelt på 1000 objektklasser. Transfer learning gir oss et godt startpunkt for visuelle effekter av lavere nivå (kanter, former, tekstrurer), som vi tilpasser til den binære klassifiseringsoppgaven. Den modellen vi lagde selv initialiseres alle vekter tilfeldig uten noe forutgående kunnskap om bildedata.
Før data mates inn i modellene, omformateres de. Alle bilder skaleres til 224x224 piksler ved hjelp av bilinear interpolering, og lastes som RGB.

For de forhåndstrente modellene normaliserer vi pikselverdier med ImageNet-mean(0.485, 0.456, 0.406) og standardavvik(0.229, 0.224, 0.225). Dette slik at inngangen matcher fordelingen modellene ble trent på. Når de trenes bruker vi data-augmentering for å gi modellene flere “syn” av hvert bilde, slik at vi reduserer overtilpasning. Standard-augmenteringen inkluderer tilfeldig resized crop med skala 0.8-1.0 og tilfeldig horisontal flipping. Når vi testet modellen vi hadde laget selv eksperimenterte vi med mer omfattende augmenteringsstrategier (colour jitter, random erasing, rotasjon, grayscale-konvertering). Disse blir videre diskutert i evalueringsdelen av rapporten.

Implementering av modell

Vi valgte å sammenligne tre fundamentalt ulike arkitekturfamilier: en klassisk residual CNN (ResNet-50), en tett koblet CNN (DenseNet-121) og en transformer-basert modell (ViT-B/16) (Dosovitskiy m.fl., 2021). Disse representerer tre toneangivende designparadigmer i moderne datamaskinsyn, og de er samtidig godt publiserte og lett tilgjengelige via Python-biblioteket "timm", noe som reduserer risikoen for at feil i implementasjonen påvirker resultatene. I tillegg tilpasser og trener vi vår egen ResNet-50 fra bunnen av, med tilfeldig initialiserte vekter og én arkitekturendring: vi bytter ut alle ReLU-aktiveringer i backbonen med GELU.

Arkitektur

ReLU og GELU er to ulike aktiveringsfunksjoner som bestemmer hvordan hver nevron reagerer på signalet det mottar. ReLU er enklest og raskest, den kutter alle negative verdier til null og slipper positive verdier gjennom uendret. GELU (Hendrycks & Gimpel, 2016) er en mykere variant som ikke kutter de negative verdiene helt bort, men slipper en liten del av dem igjennom slik at overgangen rundt null blir glattere. Dette gjør treningen mer stabil fordi gradientene ikke forsvinner brått, og det unngår det såkalte "døde ReLU"-problemet der nevroner med negativ input aldri får gradient og slutter å lære. Dette er særlig viktig når vi trener fra tilfeldige vekter uten ImageNet-forhåndstrening som startpunkt. GELU er også aktiveringen som brukes internt i ViT, noe som gir et naturlig sammenligningsgrunnlag mellom scratch-ResNet og de forhåndstrente modellene.

ResNet-50 (He m.fl., 2016) er et 50-lags konvolusjonsnettverk der hvert lag bygger videre på resultatet fra laget foran. Det spesielle med ResNet er de såkalte skip connections, eller snarveier, som lar informasjon hoppe over flere lag om gangen. Før denne idéen kom, var det nesten umulig å trene svært dype nettverk. Læresignalene

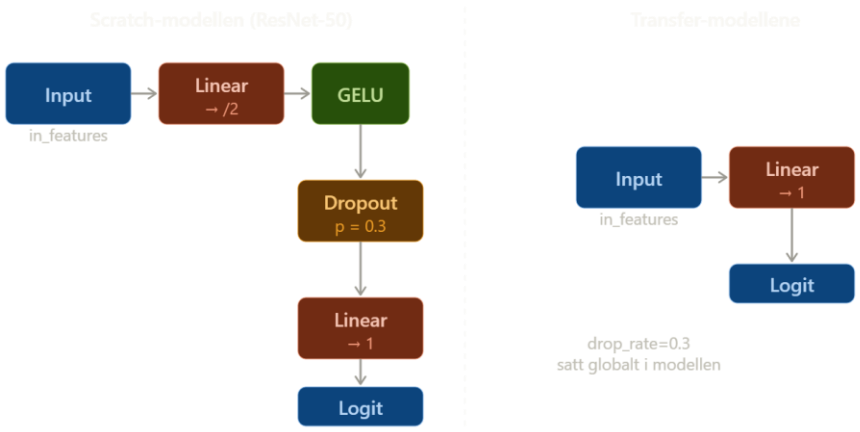
(gradientene) ble mindre og mindre for hvert lag bakover, og de innerste lagene fikk aldri beskjed om hva de burde endre. Snarveiene løser dette og fungerer samtidig som en innebygd stabilisator for treningen. ResNet-50 er en naturlig basislinje fordi den er godt studert, raskere å trene enn dypere varianter, og har blitt den gjeldende standarden for transfer learning på bildeoppgaver av ImageNet-størrelse.

DenseNet-121 (Huang m.fl., 2017) tar idéen om informasjonsflyt enda lenger. I et vanlig nettverk får hvert lag bare input fra laget rett foran seg, men i DenseNet får hvert lag input fra alle tidligere lag samtidig. Dermed kan nettverket gjenbruke mønstre det allerede har lært, i stedet for å måtte lære dem på nytt i hvert eneste lag. Resultatet er et kompakt nettverk med færre parametere enn et tilsvarende ResNet, og med god informasjonsflyt gjennom hele dybden.

ViT-B/16 (Dosovitskiy m.fl., 2021) bryter helt med konvolusjonstilnærmingen. I stedet for å gli en liten filterrute over bildet, deler modellen bildet opp i små biter på 16 × 16 piksler og behandler hver bit som et "token" på samme måte som en språkmodell behandler ord i en setning. Den sentrale mekanismen heter self-attention, og den lar modellen sammenligne alle deler av bildet med hverandre samtidig. Der et konvolusjonsnettverk starter med å se lokale mønstre og gradvis bygger opp et globalt bilde gjennom mange lag, kan en transformer se hele bildet helhetlig allerede fra første lag i treningen. Det gjør den teoretisk sett særlig godt egnet til å fange globale inkonsistenser, som ofte er signaturen på et AI-generert bilde.

Klassifiseringshode

For scratch-modellen bygde vi et eget klassifiseringshode som erstatter det originale (den delen av nettverket som gjør om de lærte representasjonene til en endelig prediksjon):
Linear(in_features, in_features // 2) → GELU → Dropout(0.3) → Linear(in_features // 2, 1).
Det er et lite to-lags nettverk som krymper representasjonen til halvparten av størrelsen, sender den gjennom en GELU-aktivering, slår tilfeldig av 30 % av nevronene for å hindre overtilpasning (dropout), og presser alt ned til ett enkelt tall, kjent som en logit.
For de tre forhåndstrente modellene beholdt vi timmens (python library) standard klassifiseringshode, som er ett enkelt lineært lag: Linear(in_features, 1), med drop_rate=0.3 satt globalt i modellen.



[Bilde 2] Illustrasjon av de to forskjellige klassifiseringshodene benyttet.

Treningsstrategi

Felles for alle fire er tapsfunksjonen *BCEWithLogitsLoss*, som kombinerer sigmoid og binær kryss-entropi i én numerisk stabil operasjon og er standardvalget for binære klassifikasjonsoppgaver. Ved inferens sendes logit-verdien gjennom en sigmoid-funksjon som gjør et hvilket som helst tall om til en verdi mellom 0 og 1 , som gir oss sannsynligheten for at bildet er AI-generert. Hvis verdien er over 0.5 svarer modellen "AI-generated".

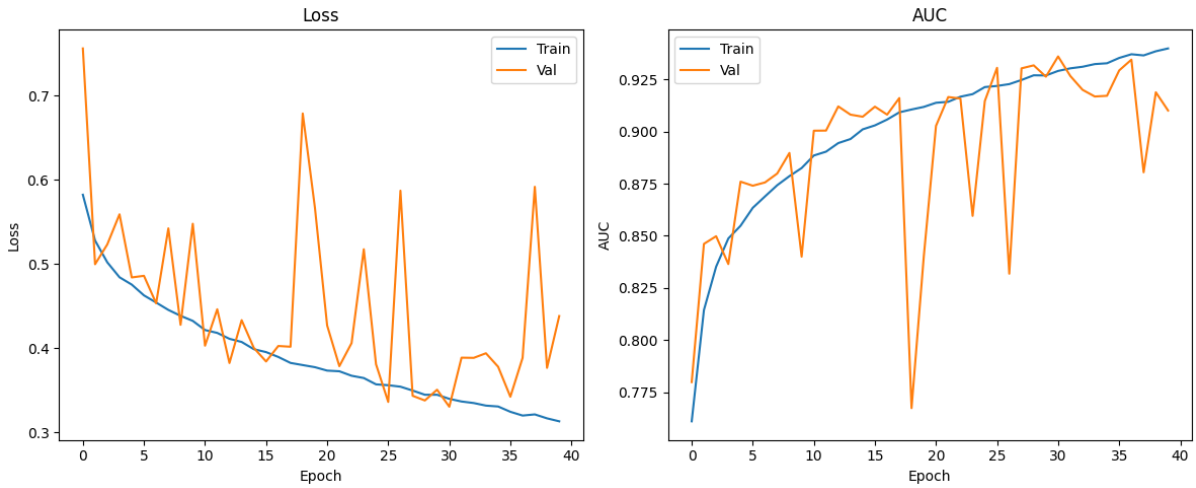
For de tre forhåndstrente modellene brukte vi en to-fase treningsstrategi som er etablert praksis for transfer learning (Yosinski m.fl., 2014). I fase 1 fryser vi hele backbonen og trener kun klassifiseringshodet i fem epoker med en relativt høy læringsrate (lr = 1e-3), slik at hodet raskt får et godt utgangspunkt uten at de ImageNet-trente representasjonene forstyrres. I fase 2 låser vi opp alle vektene og finjusterer hele nettverket i inntil 30 epoker med en mye lavere læringsrate (lr = 1e-5). En for høy læringsrate i fase 2 ville skrevet over de forhåndstrente trekkene raskere enn nettverket rakk å lære noe nytt. En scheduler kalt CosineAnnealingLR reduserer læringsraten gradvis mot null gjennom fasen og hjelper modellen å konvergere mot et stabilt minimum.

For scratch-modellen kjører vi én lang fase med samme lave læringsrate (lr = 1e-5) i inntil 80 epoker. Tidligere versjoner forsøkte høyere læringsrater og label smoothing (der målverdiene 0 og 1 dyttes litt inn mot midten for å hindre at modellen blir overtrygg), men både konvergens og evnen til å skille klassene skarpt fra hverandre ble bedre uten. Scratch-modellen bruker også en høyere tålmodighet (patience = 10) før early stopping slår inn, fordi trening fra tilfeldige vekter naturlig er mer ujevn enn finjustering.

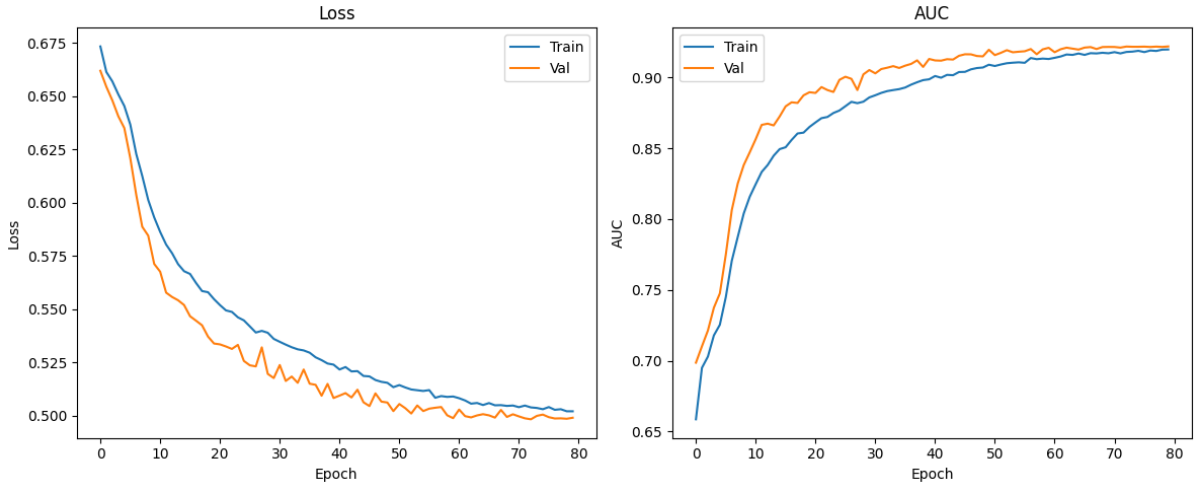
Hyperparameter og iterasjon

Øvrige hyperparametere er felles for alle fire eksperimentene: Adam-optimizer med weight decay 1e-4, batch size 64, og cosine annealing som læringsrateplan. Under trening logger vi trenings- og valideringstap og AUC etter hver epoke, lagrer den modellen med høyest validerings-AUC som "best", og skriver et fullstendig gjenopptakbart sjekkpunkt hver epoke. Alle eksperimenter bruker seed 42 for reproduserbar datasplitt.

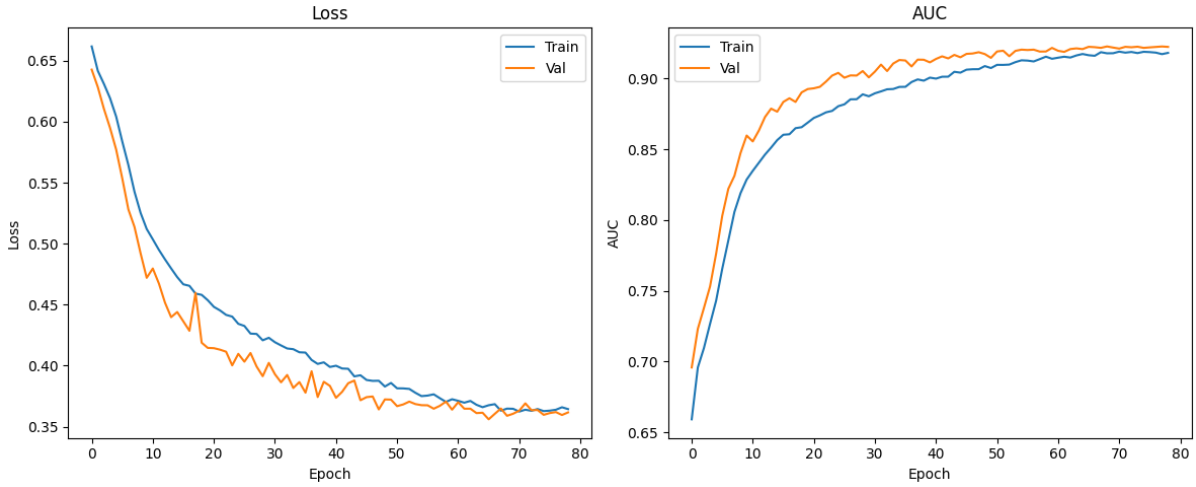
Scratch-modellen ble itert gjennom fire versjoner: v1 var en ren basislinje, v2 innførte GELU og label smoothing, v3 la til mild augmentering (color jitter og random erasing), og v4 droppet label smoothing og økte patience til 10 etter at label smoothing ga en systematisk nedgang i ytelse.



[Bilde 3] Grafer som viser trainloss og AUC på første versjon av scratch-modellen.



[Bilde 4] Grafer som viser trainloss og AUC på tredje versjon av scratch-modellen.



[Bilde 5] Graf som viser trainloss og AUC på V4 av scratch-modellen.

Ser man på basismodellen er valideringstapet svært ustabilt med store pigger og tilsvarende dykk i validerings-AUC ned mot 0.775, noe som tyder på at modellen sliter med å generalisere stabilt uten augmentering og med høyere læringsrate. I v3, der augmentering og label smoothing ble introdusert, stabiliserte treningen seg betydelig, men loss flater ut rundt 0.50, man kan anta det er fordi label smoothing kunstig hever tapsminimum ved å hindre modellen i å lære harde 0 og 1 som mål. I v4 ble label smoothing fjernet og patience økt til 10. Resultatet er det mest stabile og laveste tapskonvergensen av de tre, med loss ned mot 0.36 og en jevn AUC-kurve gjennom alle 79 epoker. Grafene viser tydelig at kombinasjonen av augmentering og fjerning av label smoothing var avgjørende for å oppnå stabil og god konvergens.

Evaluering

For å vurdere modellene brukte vi metrikkene definert i problembeskrivelsen: AUC (*Area Under the ROC Curve*), accuracy, precision, recall og F1-score. Resultatene ble beregnet på det originale Kaggle-testsettet (12 000 bilder, 6 000 ekte + 6 000 AI-genererte), som er fullstendig holdt utenfor både treningen og modellseleksjonen. Det betyr at tallene gir et strengt og upartisk estimat på hvordan modellen vil prestere på nye, usette bilder. Den endelige modellen som evalueres er den vi lagret som "best" under trening, det vil si den med høyest validerings-AUC gjennom kjøringen.

Accuracy er meningsfull her fordi datasettet er balansert (1:1). Precision og recall skiller mellom de to feiltypene som har ulike konsekvenser i praksis: en falsk positiv diskrediterer en ekte bildekilde, mens en falsk negativ lar et AI-bilde slippe gjennom

uoppdaget. F1 balanserer disse to. AUC velges som hovedmål fordi det er tersklingsuavhengig og egner seg godt til å sammenligne modeller mot hverandre.

Begge suksesskriteriene vi satte i problembeskrivelsen ble innfridd. Scratch-modellen oppnådde en test-AUC på 0.9234 (mål: ≥ 0.90), og de tre forhåndstrente modellene leverte alle godt over 0.95 i AUC (mål: ≥ 0.95 for minst én av dem). Tabell 1 oppsummerer resultatene.

Modell	**AUC**	**Accuracy**	**Precision**	**Recall**	**F1**	**Trenings tid**	**Epoker**
ResNet-50 (scratch, Gelu)	0.923	84.40%	81.87%	88.37%	0.850	27,7 timer	79
ResNet-50 (transfer)	0.978	91.70%	89.14%	94.97%	0.919	8.8 timer	35
DenseNet-121 (transfer)	0.986	94.12%	93.46%	94.87%	0.941	11.7 timer	32
ViT-B/16 (transfer)	0.995	96.86%	98.02%	95.65%	0.968	24.3 timer	17

Tabell 2: Testresultater for de fire modellene på Kaggle-testsettet (12 000 bilder).

ViT-B/16 var klart best på tvers av alle metrikkene. Den traff særlig høyt på precision (98.02 %), som betyr at av alle bildene den merket som AI-generert var hele 98 % faktisk AI-generert. Med andre ord gir modellen svært få falske alarmer. ResNet-50 i transfer learning-konfigurasjon var den svakeste av de tre forhåndstrente, særlig fordi den hadde lavere precision (89.14 %). DenseNet-121 lå et lite stykke bak ViT på alle metrikkene, men brukte vesentlig mindre treningstid (~11.7 timer mot ~24.3 timer).

Vår egentrente scratch-modell endte på AUC 0.9234, et godt stykke under de tre forhåndstrente, men trygt over den selvsatte målgrensen. Avstanden er ventet og illustrerer det sentrale poenget med transfer learning: ImageNet-forhåndstrening gir nettverket en gratis "synsforståelse" gjennom 1.2 millioner bilder fra hundrevis av kategorier, mens vi i scratch-modellen må prøve å lære oss alt sammen på 38 400 treningsbilder fra to klasser alene. At scratch-modellen klarer å levere AUC over 0.92 og F1 over 0.85 er et tegn på at signalet i datasettet er sterkt og at arkitekturen og treningsprosedyren fungerer, men gapet mot ViT (0.9958) viser hvor mye gratis representasjonslæring fra ImageNet kan bidra med.

For å vurdere kvaliteten på prediksjonene utover de aggregerte metrikkene brukte vi to standardvisualiseringer. ROC-kurven plotter andelen riktig oppdagede AI-bilder mot andelen feilaktig flaggede ekte bilder ved alle mulige terskler, og er grunnlaget for AUC-tallet. Confusion matrix viser hvor mange bilder som havner i hver av de fire mulige kombinasjonene (faktisk klasse × predikert klasse) ved den valgte terskelen 0.5.

Model: Resnet-50	Predikert Real:	Predikert Fake:
Faktisk Real:	5308	692
Faktisk Fake:	302	5698
Model: DenseNet-121		
Faktisk Real:	5602	398
Faktisk Fake:	308	5692
Model: ViT-B/16		
Faktisk Real:	5884	116
Faktisk Fake:	261	5739
Model: ResNet50 (scratch + GELU)		
Faktisk Real:	4826	1174
Faktisk Fake:	698	5302

Tabell 3: Confusion matrix for de fire modellene for kaggle-testsettet.

På tvers av modellene ser vi at ViT-B/16 produserer langt færre falske positive (ekte bilder feilmerket som AI) enn de øvrige, mens ResNet-50 i transfer-konfigurasjon har klart flest av disse. Begge typer feil er problematiske i praksis, men de har ulike konsekvenser: en falsk positiv (et ekte fotografi merkes som AI) kan ufortjent diskreditere en bildekilde, mens en falsk negativ (et AI-bilde slipper gjennom) svekker tilliten til verktøyet og lar desinformasjon spre seg.

Vi vurderte også å bruke forklaringsverktøy som Grad-CAM (Selvaraju m.fl., 2017) for å se hvilke bildeområder modellen baserer prediksjonen sin på, men dette ble ikke gjennomført innenfor prosjektets tidsramme. Det er en begrensning vi reflekterer over i konklusjonen.

Et viktig spørsmål er om modellene er trent til sitt potensial. ViT-B/16 konvergerte raskest av de fire (early stopping slo inn etter 17 epoker), men hadde lengst treningstid per epoke fordi self-attention er beregningsmessig dyr. ResNet-50 og DenseNet-121 brukte henholdsvis 35 og 32 epoker. Scratch-modellen kjørte derimot helt til epoke 79 av 80; beste validerings-AUC var 0.9223, og test-AUC på Kaggle-settet ble 0.9234. At de to tallene er praktisk talt identiske er et godt tegn på at modellen generaliserer, og det utelukker betydelig overtilpasning. Tendensen tyder likevel på at modellen fortsatt var i ferd med å forbedre seg da kjøringen tok slutt. Vi tror derfor at scratch-modellen kunne nådd noe høyere ytelse med flere epoker eller en mer aggressiv læringsrateplan, men forholdet mellom treningskostnad og forventet ytelsesgevinst gjorde at vi ikke prioriterte å forfølge dette videre.

Utover modellytelsen rapporterer vi tre praktiske metrikker som er viktige for deployment. Treningstid varierer fra ~8.8 timer (ResNet-50 transfer) til ~27.7 timer (scratch ResNet-50, totalt på alle versjonene brukte den ~65 timer) på én GPU. Scratch-modellen er paradoksalt nok dyrere å trene enn ViT, fordi den må lære representasjonene fra null over 79 epoker. Modellstørrelsen ligger i området 8–86 millioner parametere, ViT-B/16 er størst, DenseNet-121 minst, og ResNet-50 ligger imellom. Inferens-tid på CPU er praktisk talt momentan for alle modellene (godt under ett sekund per bilde), og er derfor ikke en flaskehals i Streamlit-appen. Disse tallene forteller oss at hvis man er villig til å tåle lengre trening og litt høyere lagringsplass, gir ViT-B/16 best totalpakke for denne oppgaven, mens DenseNet-121 fremstår som et fornuftig kompromiss mellom ytelse og ressursbruk.

Deployment

Vi har deployet modellene som en interaktiv nettapplikasjon. Nettapplikasjonen ble bygget med Streamlit, som er et rammeverk for Python som brukes for å lage enkle webapper for maskinlæring. Applikasjonen ble publisert på Hugging Face Spaces, som er en plattform hvor man kan hoste maskinlæringsdemoen gratis. Dette gjør at hvem som helst kan teste applikasjonen direkte i nettleseren, uten å laste ned noe lokalt. Appen er tilgjengelig på: [https://huggingface.co/spaces/Joergenator/imagerecogniser]. Formålet ved å deploye har vært både for å demonstrere at modellene fungerer "end to end" på reelle og ukjente brukerbilder. Samtidig for å gjøre det enkelt for medstudenter og sensorer å verifisere resultatet selv.

Når man går inn på nettsiden møter brukeren et enkelt grensesnitt. En nedtrekksmeny lar dem velge mellom de fire modellene. Til filopplastning aksepteres JPG, PNG og WebP. Når et bilde lastes opp viser appen både selve bildet, og klassifikasjonsresultatet hvor det enten står "Real" eller "AI-generated". Det vises også sannsynligheten for at bildet er AI-generert som både prosent og en visuell progresjonsindikator. Det er også et felt som kan utvides med tittelen "What does this number mean?", som forklarer logit-verdien og terskelen på 0,5. Dette er bare for de brukerne som vil vite mer om hvordan modellen fungerer.

Rammeverket viste seg å være godt egnet til formålet. Streamlit krevde minimalt med oppstartskode. Det var innebygd caching av modellinnlastningen via @st.cache_resource. Slik at vektene bare lastes en gang per kjøring i stedet for ved hver opplastning. Streamlit spilte sømløst sammen med PyTorch-modellene våre slik at integrasjonen mellom frontend og backend var enkel, og modellene kunne brukes direkte. Vektene på modellene er lagret på Hugging Face Hub i et separat repository fra koden og lastes ned automatisk ved første bruk. For deretter å caches lokalt på disk. Ved å holde dette separert holder vi GitHub-repoet (https://github.com/Joergenator/ImageRecogniser) rent, med liten kode som er lettleselig, samtidig som Hugging Face tar seg av båndbredde og lagring av store filer. Som for eksempel vektene.

For vedlikehold og monitorering har vi ikke per nå implementert noen form for automatisk feilrapportering. Vi ser ikke dette som nødvendig på grunn av at prosjektet er en demonstrasjon for et prosjekt via skolen. Hvis dette var et reelt deployment scenario ville vi ha logget prediksjoner og opplastninger anonymt med samtykke, slik at det var mulig å følge drift over tid. Dette er spesielt viktig ved maskinlæring, da modellene våre raskt blir mindre treffsikker. Når det gjelder AI-generering av bilder blir modellene som brukes til dette bedre og bedre, noe som igjen gjør det vanskeligere å

skille mellom hva som er ekte og hva som er falskt. En måte å kontrollere hvor vidt modellen vår henger med er å ha en periodisk re-evaluering på et oppdatert testsett.

Videre utvikling ville vært å kombinere flere av modellene, for å øke robustheten. Slik at utfallet kommer fra gjennomsnittet av modellene. En annen smart videreutvikling er å lage støtte for opplastning av flere bilder på samme tid. Dette slik at journalister eller lignende raskt kan gjennomgå større materiale. Integrering av en forklaringsmetode som Grad-CAM ville vært til stor hjelp med å forklare brukeren hvordan modellen kommer fram til et svar. Den viser hvilke bildeområder modellen baserer prediksjonen sin på. Dette ville styrket troverdigheten ved tilfeller der sannsynligheten ligger nær 0.5, rett på grensen mellom ekte og generert. En “Tilbakemelding” knapp som lar brukeren rapportere åpenbare klassifiseringsfeil ville vært med å gitt et grunnlag for kontinuerlig forbedring av modellen.

Konklusjon

Prosjektet hadde som mål å bygge en binær klassifikator som skiller ekte fotografier fra AI-genererte bilder, og å dokumentere en komplett pipeline fra datainnsamling og modelltrening til deployment som webapp. Begge målsetningene ble innfridd. Vi trente fire modeller under identiske datamessige forhold, oppnådde test-AUC mellom 0.9234 og 0.9958, og leverte en fungerende Streamlit-app der brukere kan teste klassifikatoren selv. Den egentrente scratch-modellen passerte den forhåndsdefinerte terskelen på AUC ≥ 0.90, og minst én forhåndstrent modell passerte AUC ≥ 0.95. Faktisk gjorde alle tre forhåndstrente modeller det.

Sett i ettertid mener vi at de valgte metrikkene var gode mål på prosjektets suksess. AUC ga et tersklingsuavhengig bilde av modellenes diskriminerende evne og var dermed nyttig for å sammenligne modeller seg imellom uavhengig av hvor vi setter beslutningsgrensen. Accuracy alene ville vært tilstrekkelig her, ettersom datasettet er balansert, men kombinasjonen av precision, recall og F1 ga oss et fint bilde av hvilke typer feil hver modell gjør, noe som er viktig i et virkelig anvendelsesscenario der falske positive og falske negative har ulike konsekvenser. Det vi kanskje skulle lagt mer vekt på, var generaliseringsmetrikker, særlig hvordan modellene presterer på AI-bilder fra generatorer som ikke var representert i treningsdataene. Slik out-of-distribution-evaluering ville gitt et mer realistisk bilde av hvordan modellen ville fungere med nye tilfeldige bilder.

Sammenligningen mellom scratch-modellen og de forhåndstrente illustrerte tydelig verdien av transfer learning. Gapet på over fem AUC-prosentpoeng mellom scratch-ResNet og transfer-ResNet, som bygger på samme grunnarkitektur og er trent på samme data, er en god demonstrasjon av hvor mye representasjonslæring fra ImageNet bidrar med. De to modellene er ikke helt identiske, ettersom scratch-varianten bruker GELU i stedet for ReLU i backbonen, men siden GELU generelt regnes som en like god eller bedre aktivering, er det rimelig å anta at aktiveringsfunksjonen ikke forklarer gapet, og at det i hovedsak skyldes fraværet av ImageNet-forhåndstrening.

Av kjente begrensninger og kilder til feil er det verdt å nevne noen. For det første er datasettet vårt produsert av et begrenset utvalg bildegeneratorer, og vi har ikke verifisert at modellen generaliserer til nyere modeller den ikke har sett under trening. For det andre gjennomførte vi ikke forklaringsmetoder som Grad-CAM, og vi vet derfor ikke med sikkerhet hvilke bildetrekk modellen faktisk reagerer på, om den fanger meningsfulle artefakter, eller om den kanskje plukker opp utilsiktede signaturer som JPEG-kompresjonsmønstre eller filtypevalg.
For det tredje oppdaget vi ved uformell testing med egne bilder at modellene konsekvent klassifiserer ekte selfier og portretter av mennesker tatt på nært hold som

AI-genererte. Ved manuell inspeksjon av fake-klassen i datasettet fant vi en sannsynlig forklaring: AI-genererte portretter og nærbilder av personer er overrepresentert i fake-klassen sammenlignet med ekte-klassen. Dette gir modellene et feil insentiv under trening. I stedet for utelukkende å lære å gjenkjenne diffusjonsartefakter, kan de ha lært at "nærportrett av person" i seg selv er et signal for AI-generering.
Resultatet er at ekte selfier klassifiseres som fake ikke fordi de inneholder artefakter, men fordi motivtypen korrelerer med fake-klassen i treningsdataene. Dette er et eksempel på spurious correlations, og er en viktig påminnelse om at høy AUC på testsettet ikke nødvendigvis betyr at modellen har lært de riktige trekkene, særlig dersom test- og treningssett deler den samme skjevheten.

Hvis vi skulle gjøre prosjektet på nytt, ville vi prioritert fire ting.
(i) et mer mangfoldig datasett med bilder fra flere ulike generatorer, og med balansert motivfordeling mellom ekte og fake slik at modellen ikke kan lære motivtype som snarvei.
(ii) Tidlig implementering av Grad-CAM eller en lignende forklaringsmetode for å avdekke nettopp slike spurious correlations underveis i treningen.
(iii) Et grundigere hyperparametersøk for scratch-modellen, særlig på læringsrate og epokeplan, ettersom resultatene tyder på at vi ikke utnyttet potensialet fullt ut.
(iv) Systematisk testing på tvers av ulike motivkategorier, som portretter, landskap, objekter og abstrakte bilder, for å avdekke svakheter som ikke fanges av en samlet AUC-måling.
Likevel er vi godt fornøyd med sluttresultatet. Vi har laget en reproduserbar og dokumentert pipeline, en modell som oppfyller egne mål, og fått til å deploye den slik at det er enkelt å teste ut resultatet.

Kilder

AI or not *(u.å.). AI Detection for Truth Seekers*. https://www.aiornot.com/

Bird, J. J. & Lotfi, A. (2023). *CIFAKE: Image classification and explainable identification of AI-generated synthetic images*. arXiv. https://arxiv.org/abs/2303.14126

Corvi, R., Cozzolino, D., Zingarini, G., Poggi, G., Nagano, K. & Verdoliva, L. (2023). *On the detection of synthetic images generated by diffusion models*. IEEE Xplore. https://doi.org/10.1109/ICASSP49357.2023.10095167

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. & Houlsby, N. (2021). *An image is worth 16×16 words: Transformers for image recognition at scale*. Arxiv. https://arxiv.org/abs/2010.11929

He, K., Zhang, X., Ren, S. & Sun, J. (2016). *Deep residual learning for image recognition*. IEEE. https://doi.org/10.1109/CVPR.2016.90

Hendrycks, D., & Gimpel, K. (2016). *Gaussian error linear units (GELUs)*. arXiv. https://arxiv.org/abs/1606.08415

Hive AI. *(u.å.). AI-generated content detection*. HIVE MODERATION. https://hivemoderation.com/ai-generated-content-detection

Huang, G., Liu, Z., Van Der Maaten, L. & Weinberger, K. Q. (2017). *Densely connected convolutional networks*. IEEE. https://doi.org/10.1109/CVPR.2017.243

Lilleslåtten, M. (2023, 2. mai). *AI-bilder stiller nye krav til norske medier - og til deg som scroller*. Universitetet i Oslo. https://www.uio.no/om/aktuelt/universitetsplassen/nyheter/2023/ai-genererte-bilder-stiller-nye-krav.html

Norum Bugge, T. (2025, 30. juli). *Test deg selv - klarer du å skille ekte bilder fra AI-bilder?* TEK.NO. https://www.tek.no/nyheter/nyhet/i/bmVBO3/klarer-du-aa-skille-ekte-bilder-fra-falske

OpenAI. (2021, 05.01). *DALL-E Opprette bilder fra tekst*. https://openai.com/nb-NO/index/dall-e/

Selvaraju, R., Cogswell, R., Das, M., Vedantam, A., Parikh, R. & Batra, D. (2019, 3. desember) *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. arXiv. https://arxiv.org/abs/1610.02391

Sensity. *(u.å.).Forensic Deepfake Detection*. https://sensity.ai/

Storehaug, J. (u.å). *AI bildegenerator test - Midjourney vs DALL-E vs Stable Diffusion m.m*. tenk digitalt. https://tenkdigitalt.no/blog/ai-bildegenerator-test-midjourney-vs-dall-e-vs-stable-diffusion-m-m/

Wang, S-Y., Wang, O., Zhang, R., Owens, A. A. & Efros, A. (2019, 23 desember). *CNN-generated images are surprisingly easy to spot... for now*. arXiv. https://arxiv.org/abs/1912.11035

XDXD_003. (2025). *FaceForensics++ Dataset (C23)*. Kaggle.

https://www.kaggle.com/datasets/xdxd003/ff-c23

Yosinski, J., Clune, J., Bengio, Y. & Lipson, H. (2014). *How transferable are features in deep neural networks?* arXiv. https://arxiv.org/abs/1411.1792

Zhang, T. (2024). *ai-generated images vs real images [Datasett]*. Kaggle. https://www.kaggle.com/datasets/tristanzhang32/ai-generated-images-vs-real-images

Bilder:

[Bilde 1] https://www.nrk.no/nordland/hvem-eier-bilder-laget-av-kunstig-intelligens_-opprop-mot-bilder-laget-av-ai-1.16220177

[Bilde 2] Egen laget modell

[Bilde 3] Graf ifra prosjektoppgaven

[Bilde 4] Graf ifra prosjektoppgaven

[Bilde 5] Graf ifra prosjektoppgaven

Tabbeliste:

Tabell 1: Forklaring av måleparameter.

Tabell 2: Testresultater for de fire modellene på Kaggle-testsettet (12 000 bilder).

Tabell 3: Confusion matrix for de fire modellene for Kaggle-testsettet.Vedlegg

Vedlegg

Vedlegg 1: Ordliste

Vedlegg 2: Egenerklæring for bruk av KI

Vedlegg 3: GitHub-link